Faculty of Computing

BSc (Hons) in Software Engineering
Year 2 Semester 2 (2026)

SE2020 - Web and Mobile Technologies Lab 7

Lab Activity: Express.js

In this lab activity, students will develop a **simple RESTful backend API** using **Node.js and Express.js** for a **Movie Ticket Booking System**.

The API will allow users to:

- Add movie ticket bookings
- View bookings
- Update booking details
- Delete bookings

This activity focuses on **practical implementation** and **understanding REST API flow**.

Lab Activity Tasks

A cinema needs a simple system to manage movie ticket bookings.
Each booking contains:

- Booking ID
- Movie name
- Customer name
- Number of tickets

You are required to build a backend API to manage these bookings.

Activity 1: Project Setup

1. Create a new folder named `movie-booking-api`
2. Initialize the project as a Node.js application
3. Install **Express.js**
4. Create a file named `server.js`

Activity 2: Express Server Setup

Configure an Express server that:

- Creates an Express application
- Enables JSON request handling
- Runs on port **3000**

Verify that the server starts successfully.

```
const express = require("express");
const app = express();

// Middleware to parse JSON request bodies
app.use(express.json());

// In-memory data store
let bookings = [];

// Start server
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Activity 3: Booking Data Storage

Create an **in-memory array** to store movie booking details.

Each booking record should include:

- Booking ID
- Movie name
- Customer name
- Ticket count

Activity 4: Add a New Booking

Create an API endpoint to:

- Accept booking details from the request body
- Add a new booking to the system
- Return a success message

Test this endpoint using Postman.

Endpoint

POST /bookings

```
app.post("/bookings", (req, res) => {
  const booking = {
    id: bookings.length + 1,
    movieName: req.body.movieName,
    customerName: req.body.customerName,
    ticketCount: req.body.ticketCount
  };

  bookings.push(booking);
  res.status(201).json({
    message: "Booking added successfully",
    booking  });
});
```

Expected Postman Body (Example)

```
{
  "movieName": "Avatar",
  "customerName": "Kamal",
  "ticketCount": 3
}
```

Activity 5: View All Bookings

Create an API endpoint to:

- Retrieve all movie bookings
- Display the data in JSON format

Endpoint

POST /bookings

```
app.get("/bookings", (req, res) => {
  res.json(bookings);
});
```

Activity 6: View a Booking by ID

Create an API endpoint to:

- Retrieve a booking using the booking ID
- Return an error message if the booking does not exist

Endpoint

GET /bookings/:id

```
app.get("/bookings/:id", (req, res) => {
  const booking = bookings.find(b => b.id == req.params.id);

  if (!booking) {
    return res.status(404).json({
      message: "Booking not found"
    });
  }

  res.json(booking);
});
```

Activity 7: Update a Booking

Create an API endpoint to:

- Update booking details using the booking ID
- Allow partial updates

Endpoint

PUT /bookings/:id

```
app.put("/bookings/:id", (req, res) => {
  const booking = bookings.find(b => b.id == req.params.id);

  if (!booking) {
    return res.status(404).json({
      message: "Booking not found"
    });
  }

  booking.movieName = req.body.movieName || booking.movieName;
  booking.customerName = req.body.customerName || booking.customerName;
  booking.ticketCount = req.body.ticketCount || booking.ticketCount;

  res.json({
    message: "Booking updated successfully",
    booking
  });
});
```

Activity 8: Delete a Booking

Create an API endpoint to:

- Delete a booking using the booking ID
- Return a confirmation message

Endpoint

DELETE /bookings/:id

```
app.delete("/bookings/:id", (req, res) => {
  const index = bookings.findIndex(b => b.id == req.params.id);

  if (index === -1) {
    return res.status(404).json({
      message: "Booking not found"
    });
  }

  bookings.splice(index, 1);
  res.json({
    message: "Booking deleted successfully"
  });
});
```